

The database needs to support high-speed write operations and ensure that the data captured is not lost under any circumstances. MQTT, Kafka and REST service components are used to ingest the data from the devices to the database.

‘Edge analytics’ performs the data translation, aggregation and filtering on the incoming data, enabling real-time decision-making at the edge. The database needs to support high-speed reads and writes with sub-millisecond latency and perform complex analytical computations on the data.

Communication layer: The communication layer supports the transmission of requests, queries, data, and results (collection and delivery). This layer acts as a bridge between distributed data sources and data storage processing units. Inter-objects and objects-to-infrastructure communication technologies are used, and interoperation guarantees are provided at upper layers.

The device manager communicates messages to the devices. The database needs to access and deliver messages to the devices with minimum latency. It consists of an IoT/cloud gateway that provides endpoints for device connectivity, facilitating bi-directional communication with the platform and enterprise systems. It also implements edge intelligence with different processing capabilities, enabling connectivity between devices, routing, filtering, and protocol and identity translation.

Query layer: The query layer handles the details of query processing and optimisation in cooperation with the federation layer and the complementary transactions layer (processing, delivery). This layer handles the components for generating, optimising, and executing queries on the IoT database level. It can be deployed at both the central and local levels.

Data layer: The data layer is the core element in the data architecture.

It performs the discovery of the data sources across the system. The catalogue of data sources, storage of data, and indexing of collected data are performed at this layer. In addition, filtering of data, pre-processing, and processing of data steps are handled at the data layer. It gathers local and autonomous data repositories to perform these activities.

A warm database is a high-speed in-memory database that reads and writes with the least latency. It provides real-time querying capabilities. This database is highly available and addresses disaster recovery.

Historical data of the IoT systems is stored in a cold database. Typical databases can be relational databases to a data lake.

Management layer: The management layer provides access and security to the various data stores in the data layer of the IoT framework. This layer consists of transaction, recovery, and security managers.

System analytics collects the data from the edge server and performs data transformation and analytics operation. The database provides the commands to perform analytical computations on the data, and stores the data as long as is required by the analytics engine.

Application layer: This layer is an important layer for IoT applications as it acts as an interface to interact with applications for end users. Devices like smartphones, computers and tablets are used as a medium to make IoT applications interactive. This layer handles the orchestration of device data collected into business processes and enterprise applications. It provides interfaces to end users, including operations.

It also implements domain-specific enterprise applications, enterprise data (includes a system of records, reference data, historical data, data warehouses, and transactional data), enterprise’s active directory (stores

user profile data), and rules or decision support systems.

Enterprise BI (business intelligence) runs reports, queries, and interfaces from historical data. The database needs to store data cost-effectively for an extended period.

Key open source databases for IoT adoption

The following are some of the top open source databases available for IoT based applications.

MongoDB: This is a powerful, flexible, open source, document-oriented and scalable NoSQL database. It supports features like indexes, range queries, sorting, aggregations, and geospatial indexes. It supports JSON to store and transmit information. JSON, being standard protocol, is a great advantage for both the web and the database. MongoDB supports a rich query language to support read and write operations (CRUD) as well as data aggregation, text search and geospatial queries.

As an example, Bosch has built its IoT suite on MongoDB.

InfluxDB: This time series database is designed to handle high write and query loads. It provides an SQL-like query language called InfluxQL for interacting with data. InfluxDB has no external dependencies and SQL-like queries are used for querying a data structure comprising measurements, series, and points. Each point consists of varied key-value pairs called field set and timestamp. Values can be 64-bit integers, 64-bit floating points, strings, and Booleans. Points are indexed by their time and tagset. InfluxDB stores data via HTTP, TCP and UDP. It has plugin support for other data ingestion protocols like Graphite and Open TSDB.

BigchainDB: This is an embedded database for IoT devices. The assets created are stored as JSON documents in BigchainDB. It helps to convert